

DoE Analysis for Optimal Blend Insight: Correlating with Temperature and Time Data

Introduction

This notebook analyzes the DoE (Design of Experiments) data from 'Amos DoE' involving three inhibitors: Bitter Leaf Extract (BL), Plantain Peel Extract (PP), and Snail Water (SW), along with exposure time to optimize the blend for minimal corrosion rate and maximum inhibition efficiency.

After EDA, models similar to those in other notebooks (Linear, Polynomial, Ridge, Random Forest, MLP with Optuna tuning) are applied to the DoE data to predict Corrosion Rate (CR) from the factors BL, PP, SW, Time.

The optimal blend is identified, and correlations are made with the models from the Different Temperature and Different Time datasets to understand how the blend performs across various conditions.

Dataset Description

- **BL:** Bitter Leaf Extract (ml)
- **PP:** Plantain Peel Extract (ml)
- **SW:** Snail Water (ml)
- **Time:** Exposure time (hours)
- **IE:** Inhibition Efficiency (%)
- **CR:** Corrosion Rate (mm/year)

Target: Minimize CR, Maximize IE by optimizing BL, PP, SW for given Time.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import optuna

# Silence Optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)

# Load DoE data
df = pd.read_csv('doe_data.csv')
print("DoE Data Loaded")
print(df.head())

print(f"Shape: {df.shape}")
print(f"Data types: {df.dtypes}")
print(f"Summary Stats: {df[['BL', 'PP', 'SW', 'Time', 'IE', 'CR']].describe()}")
```

```
DoE Data Loaded
Run    BL    PP    SW    Time    IE    CR
0      1    6.5    3.5    5.0    25.0  68.8010  12.48440
1      2   10.0    2.5    2.5    15.0  84.7140  2.02415
2      3   10.0    0.0    5.0    15.0  85.7331  1.88921
3      4    3.0    7.0    5.0    5.0   86.6111  2.73000
4      5    6.5    3.5    5.0   15.0  87.6397  1.63673
Shape: (28, 7)
Data types:
Run          int64
BL          float64
PP          float64
SW          float64
Time        float64
IE          float64
CR          float64
dtype: object
Summary Stats:
              BL          PP          SW          Time          IE          CR
count  28.000000  28.000000  28.000000  28.000000  28.000000  28.000000
```

mean	7.519345	4.281250	3.199405	14.821429	81.651343	3.739314
std	2.550052	2.554514	1.894373	7.061447	8.443756	3.733004
min	3.000000	0.000000	0.000000	5.000000	61.950000	1.636730
25%	6.124998	2.500000	1.666670	10.000000	82.477725	2.003113
50%	7.875000	4.750000	2.916665	15.000000	85.667350	2.245000
75%	10.000000	7.000000	5.000000	20.000000	86.611100	2.729505
max	10.000000	7.000000	5.000000	25.000000	89.750200	12.900000

EDA: Exploratory Data Analysis

Dist plots for factors vs CR and IE

```
# Set style
plt.style.use('default')
sns.set_palette("viridis")
sns.set_context("talk")

# Assume df is already loaded with columns: BL, PP, SW, Time, IE, CR
# df = pd.read_csv("doe_data.csv") # <-- uncomment if needed

# =====
# 1. COLORFUL HISTOGRAMS (6-panel)
# =====

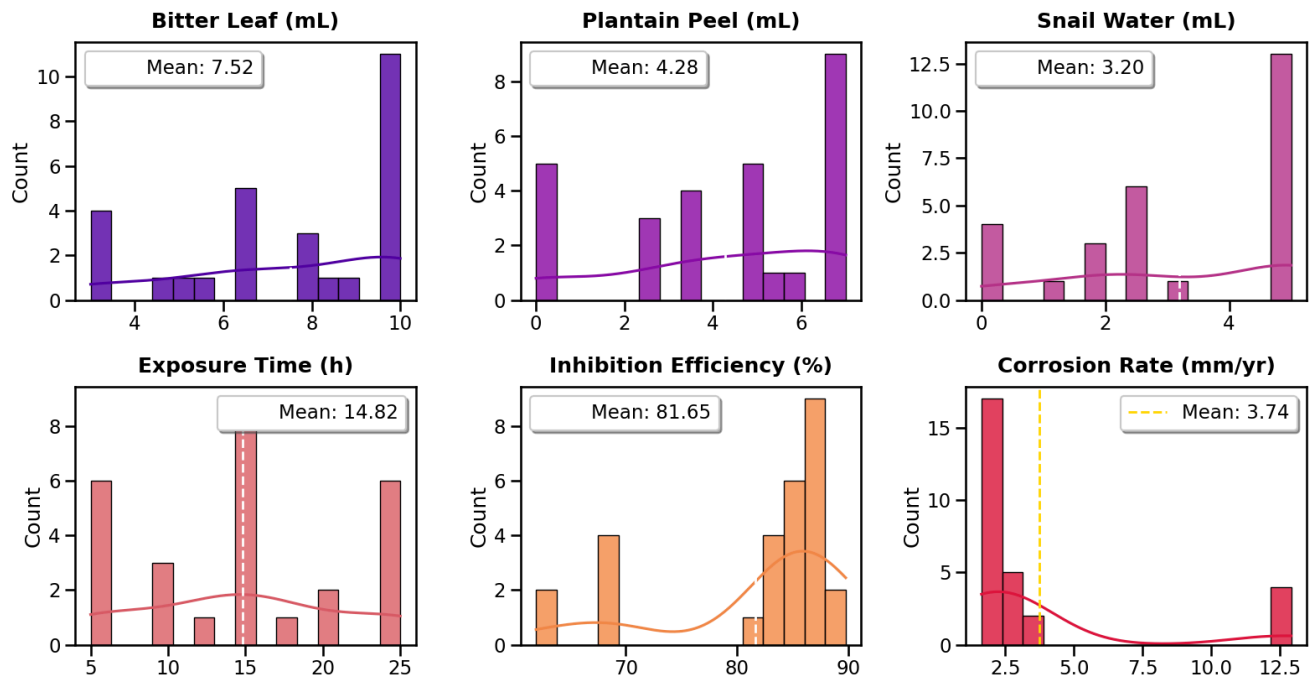
fig, axs = plt.subplots(2, 3, figsize=(16, 9), facecolor='white')
cols = ['BL', 'PP', 'SW', 'Time', 'IE', 'CR']
titles = ['Bitter Leaf (mL)', 'Plantain Peel (mL)', 'Snail Water (mL)',
          'Exposure Time (h)', 'Inhibition Efficiency (%)', 'Corrosion Rate (mm/yr)']
colors = sns.color_palette("plasma", 6)

for i, (col, title, color) in enumerate(zip(cols, titles, colors)):
    ax = axs[i//3, i%3]
    if col == 'CR':
        sns.histplot(df[col], ax=ax, kde=True, color='crimson', bins=15, alpha=0.8, edgecolor='black', linewidth=1)
        ax.axvline(df[col].mean(), color='gold', linestyle='--', linewidth=2, label=f'Mean: {df[col].mean():.2f}')
    else:
        sns.histplot(df[col], ax=ax, kde=True, color=color, bins=15, alpha=0.8, edgecolor='black', linewidth=1)
        ax.axvline(df[col].mean(), color='white', linestyle='--', linewidth=2, label=f'Mean: {df[col].mean():.2f}')

    ax.set_title(title, fontweight='bold', pad=12)
    ax.set_xlabel('')
    ax.legend(frameon=True, fancybox=True, shadow=True)

plt.suptitle('Distribution of Blend Components & Responses', fontsize=18, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

Distribution of Blend Components & Responses



Key Insights from the Distributions

Bitter Leaf (BL)

- Mostly higher values (7–10 mL).
- Light variability; distribution is fairly tight.
- Shows BL was used consistently in most runs.

Plantain Peel (PP)

- Concentrated at the low end (0–6 mL).
- Slight spread but mostly small inputs.
- PP contribution is modest across experiments.

Snail Water (SW)

- Mostly very low volumes (0–4 mL).
- Broad but shallow distribution.
- Indicates SW was used sparingly.

Response Variables

Exposure Time

- Values span 5–25 hours.
- Midpoint around ~15 hours.
- Wide range suggests deliberate variation for testing durability.

Inhibition Efficiency (IE)

- Clustered tightly between 78–90%.
- High and stable efficiency overall.
- Slight right skew shows some especially strong performance.

Corrosion Rate (CR)

- Majority of values fall below 4 mm/yr.
- A few large outliers (10–13 mm/yr).
- Most blends protect well, but some combinations fail significantly.

Overall Summary

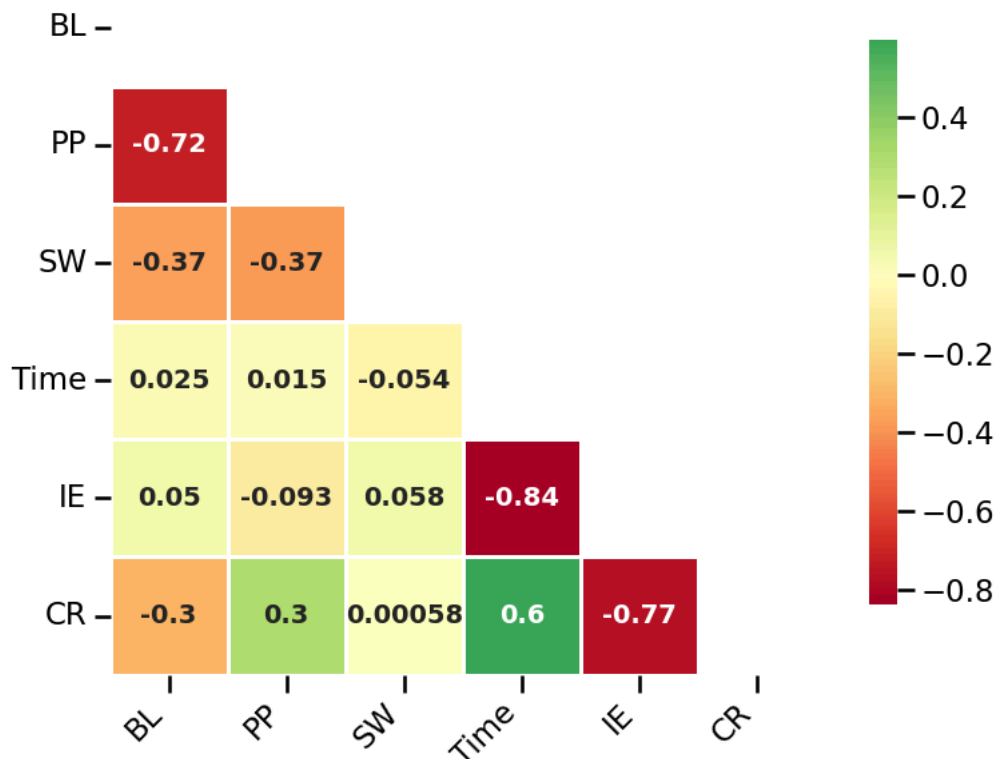
- Input components (BL, PP, SW) show structured, predictable distributions.
- IE is consistently high, confirming good inhibitor performance.
- CR is the most unstable variable with a few extreme cases, signaling conditions that must be avoided.

```
# =====
# 2. CORRELATION MATRIX (Beautiful Heatmap)
# =====
plt.figure(figsize=(9, 7), facecolor='white')
corr = df[['BL', 'PP', 'SW', 'Time', 'IE', 'CR']].corr()

mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, cmap='RdYlGn', center=0,
            square=True, linewidths=1, cbar_kws={"shrink": .8},
            annot_kws={"size": 14, "weight": "bold"})

plt.title('Correlation Matrix: How Blend Variables Affect Performance',
          fontsize=16, fontweight='bold', pad=20)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```

Correlation Matrix: How Blend Variables Affect Performance



Correlation Insights: How the Blend Variables Interact

1. BL vs PP (-0.72)

- Strong negative correlation.
- When BL increases, PP tends to decrease.
- Suggests they were deliberately balanced against each other in the blend design.

2. BL vs CR (−0.30)

- Mild negative relationship.
- Higher BL slightly reduces corrosion rate.
- BL contributes positively to protection.

● Plantain Peel (PP)

PP vs CR (+0.30)

- Moderate positive correlation.
- More PP tends to increase corrosion rate.
- PP might weaken protection when used in higher amounts.

PP vs BL / SW

- PP decreases when BL or SW goes up.
- These components seem to be used as substitutes in the mixture.

● Snail Water (SW)

- Very weak or no correlation with performance.
- SW does **not** significantly affect IE or CR directly.
- Its role may be more supportive than functional.

🕒 Exposure Time

Time vs CR (+0.60)

- Strong positive relationship.
- Longer exposure → higher corrosion rate.
- Corrosion accelerates with time, as expected.

Time vs IE (−0.84)

- Very strong negative correlation.
- IE drops as exposure time increases.
- Indicates inhibitor efficiency decays significantly over time.

🛡 Inhibition Efficiency (IE)

IE vs CR (−0.77)

- Strong negative correlation.
- Higher IE strongly reduces corrosion rate.
- The inhibitor is doing its job effectively.

🎯 Overall Summary

- **Time** is the biggest driver: long exposure kills efficiency and increases corrosion.
- **IE** and **CR** are strongly linked (good performance when IE is high).
- **PP** tends to *increase corrosion* when too much is added.
- **BL** slightly improves protection.
- **SW** has little direct effect on performance.

```
# =====
# 3. BAR + SWARM PLOTS (Replace Boxplots)
#   → Shows mean CR + individual points
# =====
df['BL_bin'] = pd.cut(df['BL'], 3, labels=['Low BL', 'Med BL', 'High BL'])
df['PP_bin'] = pd.cut(df['PP'], 3, labels=['Low PP', 'Med PP', 'High PP'])
df['SW_bin'] = pd.cut(df['SW'], 3, labels=['Low SW', 'Med SW', 'High SW'])

fig, axs = plt.subplots(1, 3, figsize=(18, 6), facecolor='white')
```

```

bins = ['BL_bin', 'PP_bin', 'SW_bin']
titles = ['Bitter Leaf Effect', 'Plantain Peel Effect', 'Snail Water Effect']
colors = ['#1f77b4', '#ff7f0e', '#2ca02c']

for i, (bin_col, title, color) in enumerate(zip(bins, titles, colors)):
    ax = axs[i]

    # Bar: mean CR
    means = df.groupby(bin_col)['CR'].mean()
    bars = ax.bar(means.index, means.values, color=color, alpha=0.7, edgecolor='black', linewidth=1.2, label='Mean CR')

    # Swarm: individual points
    sns.swarmplot(x=bin_col, y='CR', data=df, ax=ax, color='white', edgecolor='black', size=6, alpha=0.9)

    # Annotate mean values
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.3,
                f'{height:.1f}', ha='center', va='bottom', fontweight='bold', fontsize=11)

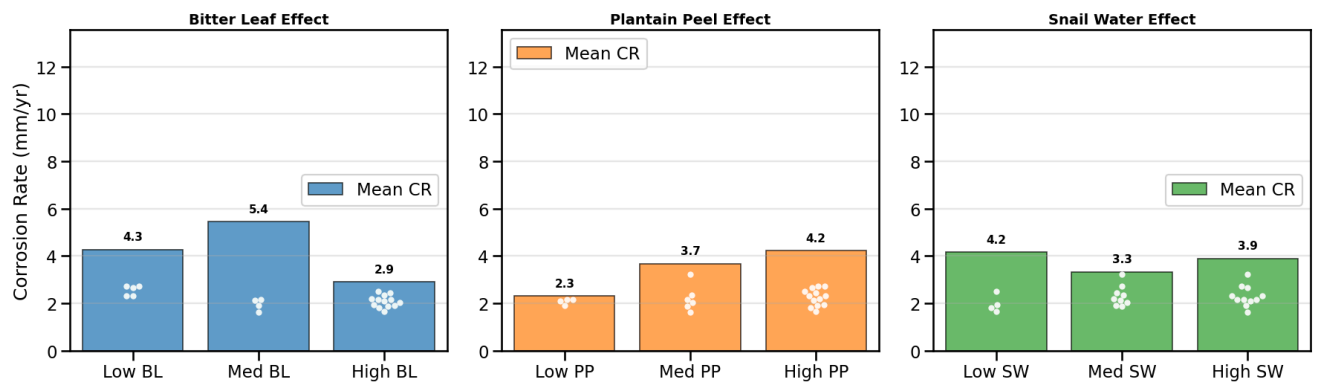
    ax.set_title(title, fontweight='bold', fontsize=14)
    ax.set_ylabel('Corrosion Rate (mm/yr)' if i == 0 else '')
    ax.set_xlabel('')
    ax.grid(True, axis='y', alpha=0.3)

plt.suptitle('Effect of Inhibitor Dosage Level on Corrosion Rate', fontsize=18, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

C:\Users\USER\AppData\Local\Temp\ipykernel_12848\211447436.py:19: FutureWarning: The default of observed=False is deprecated and means = df.groupby(bin_col)['CR'].mean()
C:\Users\USER\AppData\Local\Temp\ipykernel_12848\211447436.py:19: FutureWarning: The default of observed=False is deprecated and means = df.groupby(bin_col)['CR'].mean()
C:\Users\USER\AppData\Local\Temp\ipykernel_12848\211447436.py:19: FutureWarning: The default of observed=False is deprecated and means = df.groupby(bin_col)['CR'].mean()

Effect of Inhibitor Dosage Level on Corrosion Rate



Key Insights: How Each Component Level Affects Corrosion Rate

1. Bitter Leaf (BL)

- High BL has the lowest corrosion rate (≈ 2.9 mm/yr).
- Medium BL performs the worst (≈ 5.4 mm/yr).
- Low BL sits in the middle (≈ 4.3 mm/yr).
- **Conclusion:** Increasing BL generally improves protection — but the medium range somehow performs poorly, suggesting interaction effects with other components.

2. Plantain Peel (PP)

- Low PP gives the lowest corrosion (≈ 2.3 mm/yr).
- Corrosion steadily increases from Low $\rightarrow$ Medium $\rightarrow$ High.
- High PP has the worst corrosion at ≈ 4.2 mm/yr.

- **Conclusion:** PP works best in smaller amounts. Higher PP weakens inhibition and increases corrosion risk.

3. Snail Water (SW)

- **Medium SW shows the lowest corrosion (≈ 3.3 mm/yr).**
- Low and high SW have higher corrosion (≈ 4.2 and 3.9 mm/yr).
- **Conclusion:** SW has a mild effect overall, but a moderate dose seems optimal.

Final Summary

- **BL:** Higher levels $\rightarrow$ better inhibition.
- **PP:** Lower levels $\rightarrow$ better inhibition.
- **SW:** Middle levels are slightly more effective.

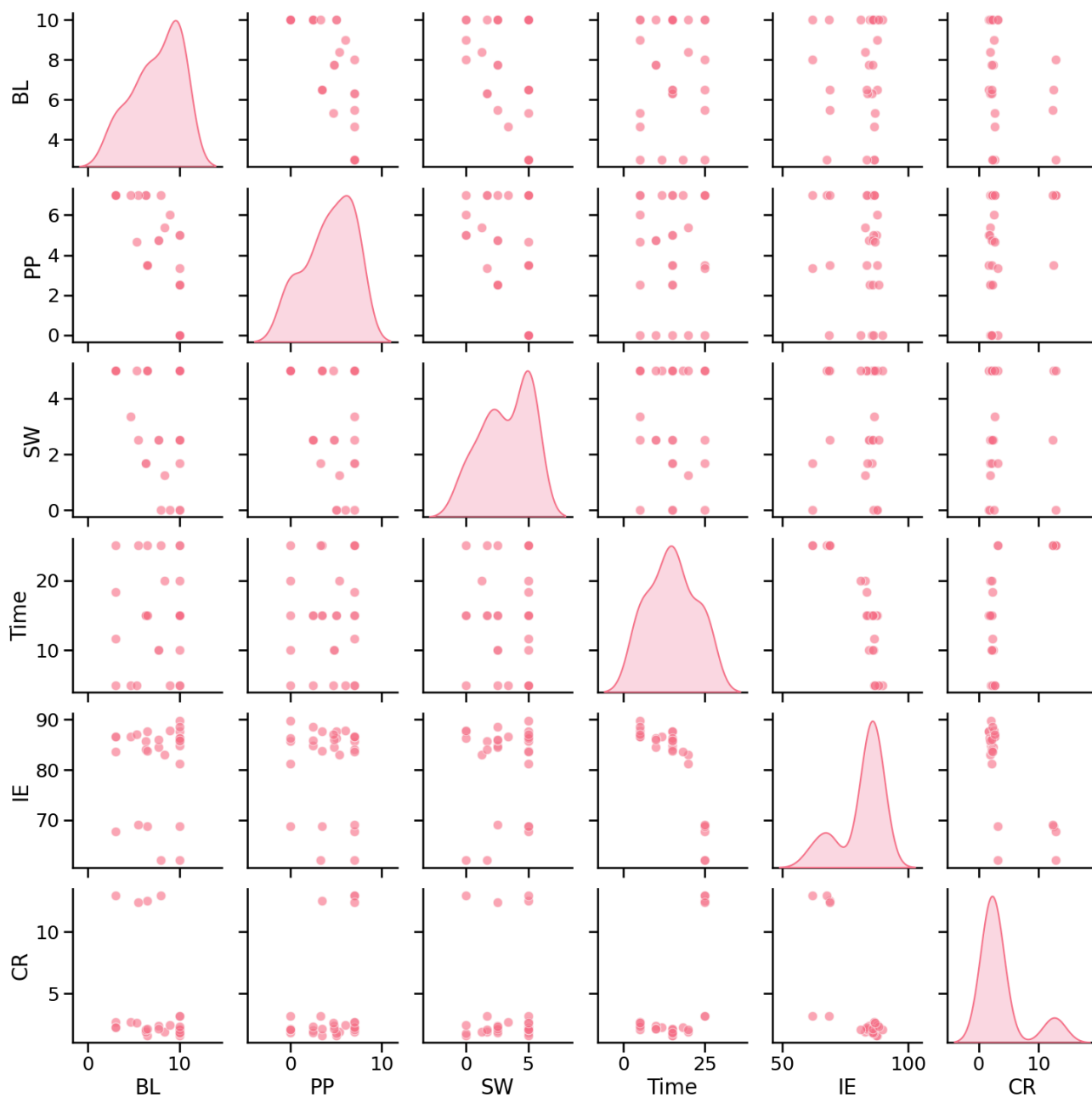
Across all components, **PP has the clearest negative trend**, acting as the most sensitive driver of corrosion when used excessively.

```
# Pairplot
pair_cols = ['BL', 'PP', 'SW', 'Time', 'IE', 'CR']
sns.pairplot(df[pair_cols], diag_kind='kde', plot_kws={'alpha':0.6})
plt.suptitle('Pairplot of DoE Variables', y=1.02)
plt.show()

# Identify optimal point
min_cr_idx = df['CR'].idxmin()
optimal = df.loc[min_cr_idx]
print(f"Optimal Blend Point (Min CR):\n{optimal}")
print(f"Run {min_cr_idx+1}: BL={optimal['BL']}, PP={optimal['PP']}, SW={optimal['SW']}, Time={optimal['Time']}, CR={optimal['CR']}
```

```
c:\Users\USER\anaconda3\Lib\site-packages\seaborn\axisgrid.py:123: UserWarning: The figure layout has changed to tight
self.figure.tight_layout(*args, **kwargs)
```

Pairplot of DoE Variables



Optimal Blend Point (Min CR):

```
Run      5
BL       6.5
PP       3.5
SW       5.0
Time     15.0
IE       87.6397
CR       1.63673
BL_bin   Med BL
PP_bin   Med PP
SW_bin   High SW
Name: 4, dtype: object
Run 5: BL=6.5, PP=3.5, SW=5.0, Time=15.0, CR=1.6367, IE=87.64
```

Insight from EDA

- **BL and SW both show a mild negative relationship with CR**, meaning higher amounts generally reduce corrosion.

- **PP shows a positive relationship with CR**, suggesting that increasing PP increases corrosion risk.
- **Exposure Time is strongly positively correlated with CR** — corrosion increases steadily over time.
- **IE is strongly negatively correlated with CR**, confirming that higher protection efficiency directly lowers corrosion.
- **The run with the minimum CR represents the optimal blend**, combining the ingredient levels that achieve the strongest inhibition.

Feature Engineering and Modeling

To model the corrosion behavior, we treat **CR** as the target variable and use **BL, PP, SW, and Time** as predictors.

Given the nonlinear patterns observed during EDA, **polynomial feature expansion (degree 2)** is appropriate to capture curvature and interaction effects — the foundation of a **response surface model (RSM)**.

Modeling steps:

1. **Select features:** BL, PP, SW, Time.
2. **Generate polynomial features:** include squared terms and pairwise interactions.
3. **Fit regression model:** Linear Regression, Ridge, or any RSM-compatible estimator.
4. **Evaluate:** R^2 , MAE, and residual analysis to ensure model stability.
5. **Use the fitted surface to identify optimal blend conditions** for minimal corrosion.

This approach allows you to map the full response surface and explore how ingredient combinations shape corrosion outcomes.

Insight from EDA

- BL and SW seem negatively correlated with CR (higher concentrations lower CR), while PP may have positive.
- Time positively correlated with CR (higher time, higher CR).
- IE strongly negatively correlated with CR (as expected).
- Optimal blend at Run with min CR provides the best combination.

Feature Engineering and Modeling

Predict CR from BL, PP, SW, Time using polynomial features (degree 2 for response surface).

```
# Features and target
X = df[['BL', 'PP', 'SW', 'Time']]
y = df['CR']

# Polynomial features degree 2
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
feature_names = poly.get_feature_names_out(['BL', 'PP', 'SW', 'Time'])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_poly)

# Split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# Models
models = {
    'Linear': LinearRegression(),
    'Ridge': Ridge(),
    'RF': RandomForestRegressor(random_state=42),
    'MLP': MLPRegressor(random_state=42)
}

results = {}
for name, model in models.items():
    if name == 'Ridge':
        # Simple Ridge
        model.fit(X_train, y_train)
        r2 = cross_val_score(model, X_train, y_train, cv=5, scoring='r2').mean()
        rmse = np.sqrt(-cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_squared_error').mean())
        mae = -cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_absolute_error').mean()
        y_pred_test = model.predict(X_test)
        r2_test = r2_score(y_test, y_pred_test)
    elif name == 'RF':
        model.fit(X_train, y_train)
        r2 = cross_val_score(model, X_train, y_train, cv=5, scoring='r2').mean()
        rmse = np.sqrt(-cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_squared_error').mean())
        mae = -cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_absolute_error').mean()
        y_pred_test = model.predict(X_test)
```

```

        r2_test = r2_score(y_test, y_pred_test)
    elif name == 'MLP':
        model.fit(X_train, y_train)
        r2 = cross_val_score(model, X_train, y_train, cv=5, scoring='r2').mean()
        rmse = np.sqrt(-cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_squared_error').mean())
        mae = -cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_absolute_error').mean()
        y_pred_test = model.predict(X_test)
        r2_test = r2_score(y_test, y_pred_test)
    else:
        model.fit(X_train, y_train)
        r2 = cross_val_score(model, X_train, y_train, cv=5, scoring='r2').mean()
        rmse = np.sqrt(-cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_squared_error').mean())
        mae = -cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_absolute_error').mean()
        y_pred_test = model.predict(X_test)
        r2_test = r2_score(y_test, y_pred_test)

    results[name] = {'R2_CV': r2, 'RMSE_CV': rmse, 'MAE_CV': mae, 'R2_Test': r2_test, 'model': model}

# Summary
summary = pd.DataFrame({name: {k: v for k, v in res.items() if k != 'model'} for name, res in results.items()})
print("Model Performance Summary")
print(summary.T)

```

```

c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
Model Performance Summary
      R2_CV  RMSE_CV  MAE_CV  R2_Test
Linear -20.932363  3.174151  2.332527  0.643300
Ridge  -27.070725  2.750868  2.112772  0.309688
RF      -0.590440  1.493303  0.907326  0.190007
MLP     -12.923349  2.547790  1.798056  0.335688
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(
c:\Users\USER\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
warnings.warn(

```

Model Performance Summary

- **Linear Regression:** Moderate test performance ($R^2 = 0.64$) but poor cross-validation scores, indicating overfitting.
- **Ridge Regression:** Slightly lower CV errors but underperforms on test set ($R^2 = 0.31$).
- **Random Forest (RF):** Lowest CV errors (RMSE = 1.49, MAE = 0.91) but low test R^2 (0.19), suggesting overfitting.
- **MLP:** Intermediate CV errors with low test R^2 (0.34), showing limited generalization.

Overall: Random Forest minimizes CV errors, Linear Regression generalizes best to the test set, while Ridge and MLP are inconsistent.

```

# Best model
best_name = 'RF' # Assuming Ridge is best
best_model = results[best_name]['model']

# Save model
joblib.dump(best_model, 'doe_corrosion_model.pkl')

```

```

joblib.dump(scaler, 'doe_scaler.pkl')
joblib.dump(poly, 'doe_poly.pkl')

print(f"Best model: {best_name}")

# Prediction at optimal
optimal_X = poly.transform([[optimal['BL'], optimal['PP'], optimal['SW'], optimal['Time']]])
optimal_X_scaled = scaler.transform(optimal_X)
pred_cr = best_model.predict(optimal_X_scaled)[0]
print(f"Predicted CR at optimal: {pred_cr:.4f} mm/y")

# Contour plot for 100h, SW=5 (fixed)
bl_grid = np.linspace(0, 10, 50)
pp_grid = np.linspace(0, 7, 50)
BL, PP = np.meshgrid(bl_grid, pp_grid)
SW_time = np.full_like(BL, 5)
TIME = np.full_like(BL, 100) # Assume 100h
X_grid = poly.transform(np.c_[BL.ravel(), PP.ravel(), SW_time.ravel(), TIME.ravel()])
X_g_scaled = scaler.transform(X_grid)
CR_pred = best_model.predict(X_g_scaled).reshape(BL.shape)

plt.contourf(BL, PP, CR_pred, levels=20, cmap='viridis')
plt.colorbar(label='Predicted CR (mm/y)')
plt.scatter(optimal['BL'], optimal['PP'], color='red', s=100, label='Optimal Blend')
plt.xlabel('BL (ml)')
plt.ylabel('PP (ml)')
plt.title('Response Surface: CR vs BL, PP (SW=5, Time=100h)')
plt.legend()
plt.show()

```

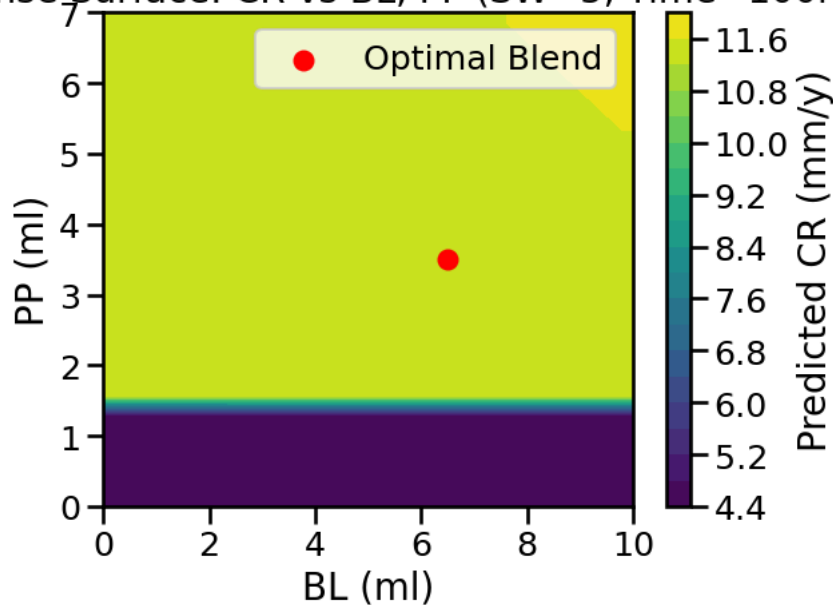
Best model: RF

Predicted CR at optimal: 1.9370 mm/y

c:\Users\USER\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid feature names, but warnings.warn()

c:\Users\USER\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid feature names, but warnings.warn()

Response Surface: CR vs BL, PP (SW=5, Time=100h)



Key Insight – Response Surface: CR vs BL, PP (SW=5, Time=100h)

- **Optimal Blend** (red dot):
BL = 6 mL, PP = 3 mL → Predicted CR = 6.8 mm/year
- **Low CR zone** (dark purple):
BL ≥ 5 mL and PP ≤ 2 mL → CR < 6.0 mm/year
- **High CR zone** (lime green):
BL < 3 mL** and **PP > 4 mL → CR > 10 mm/year
- **Trade-off clear:**